UpdatePilot

Software Update Intelligence
& Management Platform

A Windows-first full-stack application that discovers installed software, identifies available updates, presents trusted release information, and supports user-confirmed updates.

Project type Full-stack web application + Windows desktop agent

Target platform Windows 10/11 (subject to supported environment)

Delivery target Functional MVP · Professional UI/UX · Automated tests

Cost approach Free/open-source development stack; local-first operation

Document Information

Field	Details
Project name	UpdatePilot — Software Update Intelligence & Management Platform
Document type	Professional project proposal / requirements baseline
Target users	Individual Windows users; potential future IT teams
Primary platform	Windows-first; cross-platform support is out of scope for the MVP
Estimated duration	6 weeks for a scoped MVP, depending on team experience and environment
Budget principle	No paid APIs or subscriptions required for the core local MVP
Status	Proposed; technical choices and acceptance criteria should be confirmed before implementation

Contents

1. Executive Summary
2. Problem Statement and Objectives
3. Scope and Feature Requirements
4. UI/UX and User Journeys
5. Technical Architecture and Technology Stack
6. Functional and Non-Functional Requirements
7. Security, Privacy and Update Safety
8. Data Model and API Design
9. Development Plan and Milestones
10. Testing and Acceptance Criteria
11. Deliverables
12. Risks, Assumptions and Constraints
13. Free-Source and Budget Policy
14. Future Enhancements

1. Executive Summary

UpdatePilot is a Windows-first software update intelligence platform. It combines a professional web dashboard with a locally installed agent to discover installed applications, compare versions with trusted update sources, present available release details, and execute supported updates only after user confirmation.

The project is intended to deliver a functional application rather than a static dashboard. The web interface will manage inventory, scan and update jobs, release information, settings, and history. A local agent will perform operations that a browser cannot safely or directly perform on the user's computer.

The MVP will prioritize reliable application discovery, accurate package matching, update visibility, safe execution, verification, and clear UX states. Security insights and additional metadata will be displayed only when supported by a reliable source.

2. Problem Statement and Objectives

Users often do not know which applications are outdated, which releases are available, what has changed, or whether an update can be safely installed. UpdatePilot aims to make this information discoverable and actionable in one interface.

- Detect installed applications on a supported Windows computer.
- Identify available updates through trusted package sources.
- Display installed and available versions, release dates, changelogs and official links when available.
- Provide a polished, responsive and accessible dashboard.
- Enable user-confirmed updates for supported packages and verify the result.
- Record scan history, update outcomes, errors and relevant audit events.
- Provide automated tests, setup instructions and maintainable architecture.

3. Scope and Feature Requirements

3.1 In scope for the MVP

Module	Requirements
Overview dashboard	Application counts, available updates, last scan, agent connection state, recent activity and clear next actions.
Application inventory	Scan results; search, filters, sorting and pagination; installed version, publisher, package identity and source when available.
Update intelligence	Version comparison; release metadata and changelog links where available; explicit unknown, unmatched and unsupported states.
Update center	Per-application update preview and confirmation; progress states; safe execution; exit status capture and post-update verification.
History and reporting	Scan history, update attempts, before/after versions, failure diagnostics and exportable inventory reports.
Agent management	Secure pairing, connection status, last-seen timestamp, agent version, revocation and reconnection handling.
Settings	Theme, scan preferences, update confirmations, notification preferences and data retention settings as feasible.

3.2 Out of scope for the initial MVP

- Windows, macOS and Linux support in the same release.
- Silent unattended updates or automatic bulk updates by default.
- Guaranteed update support for every installed application.
- Paid commercial feeds, premium APIs, or mandatory paid cloud hosting.
- Enterprise fleet management, remote administration of other people's computers, or centralized organization-wide policy enforcement.
- Automatic vulnerability claims when package identity or affected-version data cannot be verified.

4. UI/UX and User Journeys

4.1 Visual direction

The interface should resemble a polished developer or IT operations product: restrained colors, strong typography, consistent spacing, clear hierarchy, compact data tables, informative empty states, accessible dialogs, and deliberate use of status colors. Provide light and dark themes, responsive layouts, keyboard navigation, visible focus, readable contrast and accessible labels.

4.2 Navigation model

Section	Primary purpose
Overview	Summary metrics, last scan, agent state and recent activity.
Applications	Searchable inventory and application detail pages.
Updates	Available updates, review details, confirm and monitor operations.
Security insights	Verified vulnerability/advisory information, with source and freshness.
History	Scan and update events, outcomes, logs and exports.
Settings & agent	Pairing, connection, scan preferences, theme and privacy settings.

4.3 Core user journeys

- First-time setup: install the agent → pair it securely → verify connectivity → run the first scan.
- Discover updates: start scan → view progress → review detected applications → inspect available updates.
- Update software: open application details → review target version and source → confirm update → monitor progress → verify version → review history.
- Recover from failure: show the failure reason and safe next steps; allow retry only when the job state and package source are safe.

4.4 UX acceptance principles

- All meaningful controls perform their stated action; no decorative or fake-success buttons.
- Loading, empty, error, success, disconnected and stale-data states are designed intentionally.
- Search, filters, sorting and pagination compose correctly.
- System-changing actions require explicit confirmation and display the target package and version.
- Security findings display source, date checked and limitations; unavailable data is clearly labelled.

5. Technical Architecture and Technology Stack

5.1 High-level architecture

React dashboard ↔ authenticated API and job orchestration ↔ PostgreSQL. The Windows agent performs local discovery and supported package-manager operations, and reports job results to the API through a narrow, authenticated channel. The backend retrieves and caches permitted release metadata from trusted sources.

Layer	Recommended choice
Web client	Next.js + TypeScript; Tailwind CSS; accessible component primitives; Lucide icons; Recharts for charts.
Backend	Node.js + TypeScript + Express; request schema validation; OpenAPI/Swagger; background job handling.
Database	PostgreSQL; Prisma ORM; migrations, indexes, constraints and retained history.
Windows agent	PowerShell for an initial prototype or .NET/Node.js for a packaged agent; WinGet integration; explicit operation allowlist.
Update sources	Windows Package Manager (WinGet) for supported installed packages; official release APIs where useful; optional public vulnerability data.
Testing	Vitest or Jest, Supertest, Playwright; unit, API integration, end-to-end and failure-path tests.
Dev tooling	Git, GitHub, ESLint, Prettier, Docker Compose where available, GitHub Actions within applicable free quotas.

5.2 Architecture rules

- The browser must not receive unrestricted shell access or administrator privileges.
- Long-running scans and updates must run as jobs with persistent status, timestamps and safe retry semantics.
- Use typed API contracts, centralized error handling, structured logs and configuration through environment variables.
- Keep package matching, version comparison, source adapters and update execution in separate modules.
- Cache external metadata, respect rate limits and handle source outages without breaking the core inventory.

6. Functional and Non-Functional Requirements

ID	Requirement	Priority
FR-01	Securely pair, identify and revoke a local agent.	Must
FR-02	Discover installed applications and persist scan results.	Must
FR-03	Match detected applications to trusted package identities.	Must
FR-04	Compare installed and available versions accurately for supported packages.	Must
FR-05	Show application details and official release metadata when available.	Must
FR-06	Search, filter, sort and paginate the inventory.	Must
FR-07	Execute a supported update only after explicit user confirmation.	Must
FR-08	Verify the resulting installed version and record outcome.	Must
FR-09	Persist scan/update history and useful diagnostic details.	Must
FR-10	Display security information only when the match and source are reliable.	Should
FR-11	Export inventory and update reports.	Should
FR-12	Provide responsive layouts, keyboard navigation and light/dark themes.	Must
FR-13	Support scheduled scans and configurable notifications.	Could
FR-14	Support other operating systems.	Future

6.1 Non-functional requirements

- **Security:** authentication, authorization, input validation, secure pairing, least privilege, safe command execution and secret protection.
- **Reliability:** tolerate disconnections, timeouts, interrupted jobs and unavailable metadata sources; never report success prematurely.
- **Performance:** paginate large inventories and use asynchronous jobs; define and measure response-time targets against test hardware.
- **Data integrity:** use unique constraints, transaction boundaries, job idempotency and historical snapshots.
- **Maintainability:** modular services, migrations, typed interfaces, API documentation and repeatable test commands.
- **Privacy:** collect only software inventory and operational metadata needed for the product; provide clear retention and deletion behavior.
- **Accessibility:** use WCAG 2.2 AA as a design target and test keyboard and screen-reader flows.

7. Security, Privacy and Update Safety

Updates are privileged, system-changing operations. The product must use a constrained agent interface and never treat web requests as shell commands.

- Use fixed, allowlisted update operations and validated package identifiers; do not accept arbitrary executable paths, command strings or untrusted arguments from the browser.
- Resolve package identity, publisher, architecture, source and target version before offering an update.
- Do not bypass installer hash/signature verification or use unsafe trust overrides.

- Require a clear confirmation for each update; bulk operations, if added, require a separate review and confirmation step.
- Use least privilege. Request elevation only when necessary, with a clear explanation and user approval.
- Protect agent pairing credentials; support revocation and rotation; authenticate and authorize every operation.
- Mitigate cross-origin request abuse, replayed jobs, unauthorized agent commands and cross-user data access.
- Record actor, device, package, requested target, timestamps, result and safe diagnostic metadata in an audit trail.
- Verify installer outcome and rescan the installed version before reporting success.
- Do not expose unrelated personal files, document contents, passwords, browsing history or other unnecessary host information.

7.1 Safe update workflow

User selects package → backend validates ownership and package identity → user reviews target and confirms → agent independently validates the approved operation → package manager executes → agent captures exit state → application is rescanned → backend stores outcome and audit record.

8. Data Model and API Design

8.1 Core data entities

Entity	Representative fields
Users	ID, authentication identity, created timestamp.
Devices	Owner, device name, OS/agent versions, last seen, status.
Agent credentials	Device, hashed credential, rotation and revocation metadata.
Applications	Device, display name, publisher, installed version, package ID, source.
Release metadata	Package ID, version, channel, release date, changelog URL, source and fetched time.
Scan jobs	Device, state, start/end times, error summary.
Update jobs	Application, target version, state, initiator, timestamps, exit code and verification result.
Audit events	Actor, device, event type, target, timestamp and safe metadata.
Security findings	Package identity, advisory ID, severity, affected range, source and last verified time.

Use foreign keys, unique constraints and indexes where appropriate. Store historical scan/version snapshots rather than overwriting all evidence. Define retention limits for detailed logs.

8.2 Proposed API endpoints

Method	Endpoint	Purpose
GET	/api/v1/dashboard	Overview statistics
GET	/api/v1/devices	List paired devices
POST	/api/v1/devices/pair	Secure agent pairing
POST	/api/v1/devices/:id/scans	Start a scan job
GET	/api/v1/scans/:id	Get scan progress
GET	/api/v1/applications	Search/filter inventory
GET	/api/v1/applications/:id	Application details
GET	/api/v1/updates	Available updates
POST	/api/v1/updates/:id/preview	Validate update proposal
POST	/api/v1/updates/:id/execute	Execute confirmed update
GET	/api/v1/jobs/:id	Job progress/result
GET	/api/v1/history	Scan/update history
GET	/api/v1/security/findings	Verified security findings

These routes are proposed contracts, not pre-existing working endpoints. All state-changing routes require authentication, authorization, schema validation, job-state checks and consistent error responses.

9. Development Plan and Milestones

Schedule	Focus	Expected outcome
Week 1	Requirements, UX and architecture	Confirm scope, user journeys, wireframes, design tokens, database ERD, API contracts and threat model.
Week 2	Core web application	Build dashboard, inventory UI, backend APIs, authentication, database migrations, filters and pagination.
Week 3	Agent and scanning	Implement secure pairing, installed-app discovery, WinGet integration, scan jobs and package matching.
Week 4	Update intelligence and execution	Implement version comparisons, release details, update previews, confirmed execution, progress and verification.
Week 5	Testing, security and UX polish	Test failure states, authorization, interrupted jobs, accessibility, responsiveness and integration.
Week 6	Release and documentation	Package agent, test on a clean Windows environment, fix defects and prepare demo and documentation.

Milestones

- **M1:** Approved UI/UX, architecture and working application shell.
- **M2:** Working inventory backed by real scan data.
- **M3:** Verified update detection and a functioning update workflow.
- **M4:** Tested, documented and packaged MVP.

10. Testing and Acceptance Criteria

The MVP is accepted only when it can be demonstrated on a real supported Windows computer. Sample data may be used in development, but it must be clearly labelled and must not be presented as live scan results.

Real functionality

- The agent discovers applications on the test computer.
- The dashboard displays actual scan results.
- Supported update comparisons are checked against real package-manager output.
- The Update action performs a real approved operation.
- The post-update scan verifies the installed version or reports verification failure.

UX

- Search, filtering, sorting, pagination and navigation work.
- Loading, empty, error, success, stale-data and disconnected-agent states are implemented.
- Responsive layouts and keyboard navigation are tested.
- Every visible control performs its documented action.

Quality and security

- Unauthorized device access and update requests are rejected.
- Failed, cancelled and interrupted jobs are handled safely.
- Unit, API integration and critical end-to-end tests pass.
- Setup, architecture, API, recovery and troubleshooting documentation is complete.

10.1 Suggested test coverage

Test type	Examples
Unit tests	Version comparison, package matching, input validation, job transitions and permissions.
API integration	Authentication, device ownership, database persistence, job lifecycle and error responses.
Agent integration	WinGet command parsing, timeouts, non-zero exit codes, unavailable packages and reconnect behavior.
End-to-end	Pair device → scan → view update → confirm → execute → verify → inspect history.
Security tests	Command injection attempts, unauthorized package IDs, replayed requests, CSRF/CORS behavior and secret leakage.

11. Deliverables

- Complete frontend and backend source code.
- Windows agent source and runnable package.
- Database schema, migrations and development seed data.
- Working integration with selected software update source(s).
- API documentation and architecture diagram.
- Automated test suite and test execution instructions.
- Environment-variable template and reproducible setup instructions.

- Security, privacy and data-retention documentation.
- UI/UX design system and key screen designs.
- User guide, known limitations and troubleshooting guide.
- Demonstration video or live demonstration.
- Release build with no known critical defects.

12. Risks, Assumptions and Constraints

Risk	Mitigation
Incomplete application matching	Use explicit package identities and show unmatched applications separately.
Unsupported updater	Mark the application unsupported; provide an official publisher link if verified; do not guess a command.
Installer failure or reboot required	Capture exit status, expose restart-required state and verify after restart when needed.
Agent disconnects	Track job state, expire stale requests and retry only idempotent steps.
External API limits	Cache metadata, respect retry headers and gracefully degrade optional information.
Administrator permissions	Use controlled elevation only when needed and clearly explain the action.
Unsafe command execution	Use allowlisted operations, validated parameters and independent agent-side verification.
Scope growth	Prioritize the MVP and defer cross-platform support, fleet management and unattended updates.

Assumptions

- The first release targets a supported Windows 10/11 environment with WinGet available or installable.
- The user has permission to install the local agent and authorize application updates.
- Version metadata and changelogs may be unavailable for some applications.
- The six-week estimate assumes a scoped MVP and a developer or small team with relevant experience; actual duration may vary.

13. Free-Source and Budget Policy

The core product must build, run and demonstrate locally without paid APIs, subscriptions or mandatory paid hosting. Use free/open-source dependencies with compatible licenses and document any restrictions.

Area	Free-source approach
Frontend	Next.js, TypeScript, Vite, Tailwind CSS
Backend	Node.js, Express, OpenAPI/Swagger
Database	PostgreSQL, Prisma ORM
Windows updates	Windows Package Manager (WinGet) for supported packages
Release information	Official publisher release pages/APIs where available; public GitHub Releases API where appropriate
Security metadata	Optional public vulnerability databases such as OSV, subject to coverage and source terms
Testing	Vitest/Jest, Supertest, Playwright

Area	Free-source approach
Local operations	Local services; Docker Compose where supported; self-hosted or local CI alternatives

Free development does not imply unlimited API access or guaranteed free cloud hosting. Verify current quotas, licensing and terms before use. The application must remain usable locally if optional external services are unavailable.

14. Future Enhancements

- Cross-platform agents for macOS and Linux.
- Scheduled scans, user-configurable notifications and maintenance windows.
- Optional multi-device management and organization accounts.
- Policy-driven update approvals for managed teams.
- Signed agent updates and a formal secure software distribution process.
- Offline scan reports and richer package-source adapters.
- More advanced security advisory matching with transparent confidence and source freshness.

Conclusion

UpdatePilot is a practical, industry-oriented full-stack project that combines web UX, local system integration, API design, database modelling, secure update execution and automated testing. A successful MVP must be judged by verified behavior on a real Windows machine, not by visual completeness alone. The recommended approach is to deliver a reliable Windows-first experience, document the limits of software-source coverage, and expand only after the core scan-to-update journey is secure and testable.

Implementation note: This document defines the proposed requirements and architecture. It does not represent an already-built or validated application. Confirm the final stack, package sources and supported Windows environments before development.